Week 3: Unsupervised Learning & Hyperparameter Tuning

1. K-Means Clustering

Unsupervised Learning

Definition: K-Means is an ML algorithm that divides data into a specific number of groups (clusters) based on similarities.

Simple Explanation: It groups similar things together automatically.

Example: A shopping mall divides customers into 3 groups:

- Group 1: Low spenders.
- Group 2: Average spenders.
- Group 3: High spenders.

How it works:

1. Choose the number of clusters (K).
2. Select cluster centers (centroids).
3. Assign data points to their nearest center.
4. Update the centers.
5. Repeat until clusters become stable.

Important Terms:

- K = Number of clusters.
- Centroid = Center of a cluster.
- Cluster = Group of similar data points.

2. Hierarchical Clustering

Unsupervised Learning

Definition: Hierarchical Clustering is an ML algorithm that groups similar data into a tree-like structure, showing relationships between groups.

Simple Explanation: It creates groups step by step, like building a family tree.

Example: Grouping animals:

- Cats and lions into one group.
- Dogs and wolves into another group.
- Then combining related groups into bigger groups.

Two Types:

1. Agglomerative (Bottom-Up): Starts with individual data points and combines them into bigger groups.
2. Divisive (Top-Down): Starts with one big group and divides it into smaller groups.

Important Term:

- Dendrogram = A tree-like diagram showing how clusters are connected.

3. Difference Between Them

K-Means	Hierarchical
Creates a fixed number of clusters (K).	Builds a hierarchy of clusters.
Uses centroids (centers).	Uses a tree-like structure.
Generally faster.	Generally slower for large datasets.
Uses cluster visualization.	Uses a dendrogram.

- K-Means = Divide data into groups.
- Hierarchical = Build groups like a family tree.

1. What is PCA?

Definition: Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique that transforms many original features into a smaller set of new features called principal components, while preserving as much of the important information as possible.

PCA makes a dataset smaller and easier to understand without losing too much useful information.

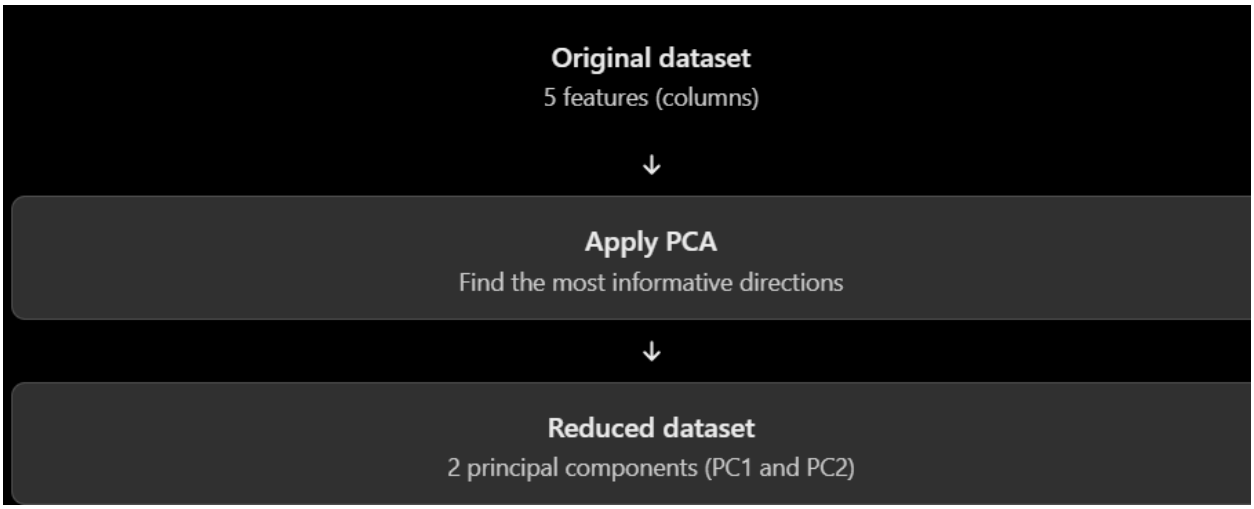
Example

Imagine you have a student dataset with these 5 features:

- Mathematics marks
- Physics marks
- Chemistry marks
- English marks
- Computer marks

If you want to visualize all these features together, it becomes difficult because a normal graph is easiest to understand in 2D or 3D.

PCA can reduce the 5 original features to 2 principal components, allowing you to display students on a 2D graph.



The result is a simpler dataset that is easier to visualize and may be faster to process. Some information can be lost during reduction.

2. Why do we use PCA?

PCA is mainly used for two important purposes.

A. Visualization

When a dataset has many features, it is difficult to visualize. PCA can reduce the data to two or three components so we can plot it on a graph and look for patterns, groups, or unusual data points.

B. Feature reduction

A dataset may contain many related or redundant features. PCA combines information from the original features into fewer new features, which can reduce complexity and improve computational efficiency.

3. Important PCA terms

Term	Simple definition	Example
Feature	A column in a dataset	Age, height, weight
Dimensionality	Number of features	10 columns = 10 dimensions
Variance	How spread out the data is	Students' marks vary from 40 to 95
Principal Component	A new feature formed from original features	PC1, PC2
PC1	Component capturing the maximum variance	Most variation in the data
PC2	Component capturing the next-highest variance, perpendicular to PC1	Additional variation
Explained Variance	Amount of variation captured by a component	PC1 captures 70%
Covariance	How two features vary together	Height and weight may increase together
Standardization	Bringing numerical features to a comparable scale	Converting height and weight to standardized values

4. How does PCA work?

PCA generally follows these steps.

Step 1: Standardize the data

Different features may have different units and ranges.

For example:

- Age: 18–60 years
- Salary: 20,000–200,000 rupees
- Height: 150–190 cm

If we apply PCA without considering these differences, features with larger numerical scales can dominate the results.

Standardization converts each feature to a scale with a mean of 0 and a standard deviation of 1.

Step 2: Calculate the covariance matrix

PCA examines how features are related to one another.

For example, height and weight may increase together. Their covariance helps PCA understand this relationship.

The covariance matrix summarizes these relationships between all pairs of features.

Step 3: Find principal components

PCA identifies new directions in which the data has the greatest variation.

- PC1: The direction with the highest variance.
- PC2: The direction with the next-highest variance, perpendicular to PC1.
- PC3: The next direction, perpendicular to the previous components, and so on.

The directions are determined using mathematical concepts called eigenvectors, while eigenvalues indicate how much variance each direction captures.

Step 4: Select the important components

PCA ranks the components by their explained variance.

For example, imagine the results are:

Component	Explained variance
PC1	70%
PC2	20%
PC3	7%
PC4	3%

If we keep PC1 and PC2, we preserve (70%+20%=90%) of the original data's variance.

We might choose two components for visualization or retain more components if we need to preserve more information.

Step 5: Transform the original data

PCA projects the original data onto the selected principal components.

For example:

- Original dataset: 4 features
- Selected components: 2

- Result: 2 new features, PC1 and PC2

The new features are combinations of the original features, not simply two original columns selected from the dataset.

5. PCA for visualization

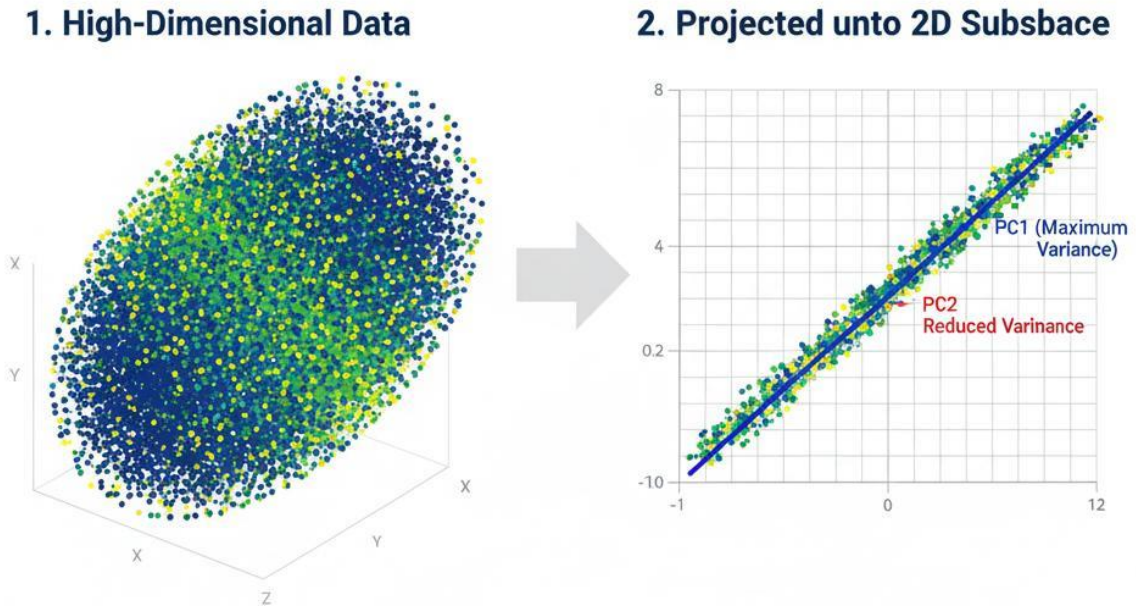
Visualization means presenting data graphically so we can understand it more easily.

Suppose a dataset contains 20 features. We cannot easily display all 20 dimensions in a normal scatter plot.

PCA can reduce these 20 features to 2 components:

- X-axis: PC1
- Y-axis: PC2

Each dot represents one observation, such as a student, customer, or flower.



6. PCA for feature reduction

Feature reduction means decreasing the number of columns used to represent a dataset.

For example:

Before PCA	After PCA
10 features Original columns	**3 components** New reduced features

The dataset has fewer dimensions, and the three components may retain most of its original variance.

Benefits include:

- Less data to process.
- Potentially faster model training.
- Less redundancy from correlated features.
- Easier visualization.

Source: https://www.geeksforgeeks.org/data-analysis/principal-component-analysis-pca/

Grid Search Cross-Validation

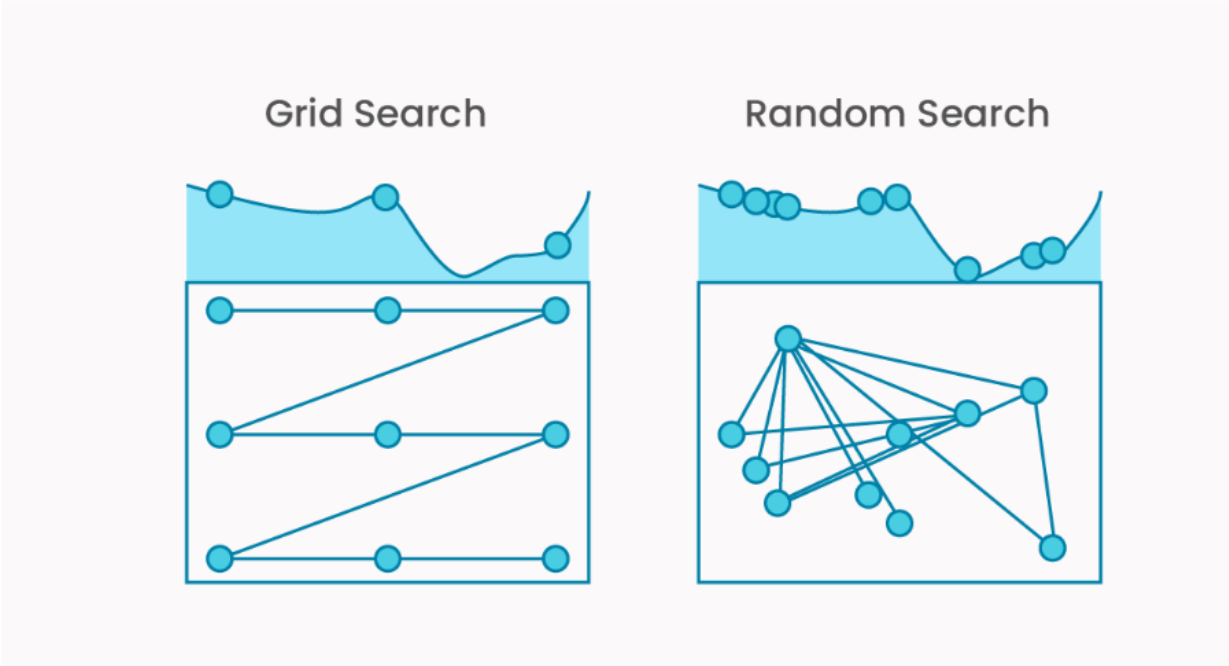
Grid search cross-validation is a technique that searches for the optimal hyperparameters of a model by evaluating the model's performance on different combinations of hyperparameter values. The idea is to define a set of hyperparameters and a range of values for each hyperparameter, and then search for the optimal combination of hyperparameters that produces the best performance on a validation set. This process is called a grid search because it searches over a grid of hyperparameters.

Random Search Cross-Validation

Random search cross-validation is a technique that searches for the optimal hyperparameters of a model by evaluating the model's performance on random combinations of hyperparameter values. The idea is to define a set of hyperparameters and a range of values for each hyperparameter, and then randomly sample values from these ranges to create different combinations of hyperparameters. This process is repeated a specified number of times, and the best combination of hyperparameters that produces the best performance on a validation set is selected.

Random search cross-validation (RandomizedSearchCV) is another powerful technique for optimizing the hyperparameters of a machine learning model. It works in a similar way to grid search cross-validation, but instead of searching over a predefined grid of hyperparameters, it samples them randomly from a distribution.

Comparison GridSearch CV and RandomSearch CV



Grid Search vs Random Search (Hyperparameter Tuning)

Both Grid Search and Random Search are techniques used to find the best hyperparameters for a machine learning model to improve its performance.

1. Grid Search

Definition: Grid Search tests every possible combination of predefined hyperparameter values to find the best combination.

- Tests all specified combinations.
- More systematic and thorough within the given grid.
- Can be computationally expensive.
- Best when the search space is small.

Example: Testing max_depth = 2, 3, 4 and n_estimators = 50, 100.

2. Random Search

Definition: Random Search selects and tests random combinations of hyperparameters from specified values or distributions.

- Tests randomly selected combinations.
- Usually faster for large search spaces.

- Does not test every possible combination.
- Best when there are many hyperparameters.

3. Main Differences

Grid Search	Random Search
Tests every combination.	Tests random combinations.
Can be slow.	Often faster.
Suitable for smaller search spaces.	Suitable for larger search spaces.
Systematic search.	Random sampling.
Can require more computation.	Computation can be controlled using n_iter.

4. Cross-Validation (CV)

Cross-validation evaluates a model on different portions of the dataset to check how well it performs on unseen data.

Both Grid Search and Random Search commonly use cross-validation to evaluate hyperparameter combinations more reliably.

Source: https://360digitmg.com/blog/comparison-gridsearchcv-and-randomsearchcv